

ReviewAgent: Confident Learning, Taxonomy-Grounded Issue Generation, and Spec-Aware Response Generation for App Reviews

Anonymous Author(s)

Abstract

App stores receive millions of reviews daily containing buried but actionable bug reports, feature requests, and performance complaints; manual triage is infeasible. Existing pipelines stop short of an actionable artifact: classifiers label and RAG generators reply, but no system produces the structured *issue specification* a developer can route into a defect tracker. LLM-based annotation, the de facto labeling method, also exhibits systematic class-collapse noise that propagates downstream uncorrected.

We present **ReviewAgent**, a four-stage pipeline composing (i) verified-anchor confident learning to correct LLM-label noise on a 215K-review corpus; (ii) aspect-grounded knowledge-graph hierarchical clustering; (iii) taxonomy-grounded **IssueSpec** generation using Zimmermann / ISO 25010 / Nielsen / user-story templates; and (iv) spec-aware retrieval-augmented response generation. Together, these stages turn a noisy review stream into a typed, developer-actionable artifact and a resolution-aware response.

Cohen’s κ against an expert gold standard rises 0.16 $\rightarrow$ 0.59 across the correction pipeline; an independently-trained classifier endorses 88.66% of cleanlab corrections. Under strict content-validity criteria, taxonomy-grounded specs reach 0.96 template-fill (Claude Opus 4.7) vs 0.53 for real GitHub issues; cross-LLM replication on Qwen2.5-3B (0.74) and Qwen2.5-1.5B (0.48) shows the score scales with capability, with the qualitative ranking preserved across a sensitivity sweep. On 400 blinded paired ratings, the spec-aware generator beats RAG-only by +2.36 quality points (paired Wilcoxon $p < 0.001$) — structure and retrieval are *complementary*, not substitutable. The pipeline yields a reproducible noise-correction recipe with third-opinion validation, an aspect-grounded clustering layer, and a structured-RAG architecture whose value over vanilla RAG is empirically established. Single-rater, single-LLM, and proof-of-concept-scale RLHF are stated as honest limitations. Artifacts: <https://github.com/Fabiha-9876/ReviewAgent>.

1 Introduction

1.1 The Problem

App-store reviews contain actionable software-engineering signal — buried bug reports, feature requests, performance complaints, usability obstacles — but no current pipeline turns them into a developer-ready *issue specification* that a defect-tracking system can consume. Google Play receives on the order of millions of new reviews per day [15, 19, 39];

the actionable subset is exactly what a development team must surface, prioritize, and respond to, but manual triage is infeasible at this volume.

Concretely, the end-to-end task is: **given a stream of unstructured reviews, produce (a) a structured issue specification (issue type, severity, affected component, reproduction steps, environment) suitable for routing into a defect tracker, and (b) a user-facing response that references the specific issue rather than a generic acknowledgement.** This is a multi-stage information-extraction cascade [25, 58], not a single classification or generation problem.

1.2 Why It Is Hard

Three properties of the data make the cascade difficult:

- **Linguistic noise.** Reviews are short, slang-heavy, multilingual, often contradictory within one sentence (“Great UI but battery drain after the update”), and rarely contain the structured fields a defect tracker expects.
- **Long-tailed, skewed class distribution.** A few classes (**praise**, **bug_report**) absorb the bulk of reviews; operationally important minority classes (**compatibility**, **performance**) are rare enough that supervised training data is hard to assemble at scale.
- **LLM bootstrap labels are biased.** Production-scale labeling now relies on LLM annotation [21, 63], but LLM annotators introduce *systematic* (not random) biases — class collapse, boundary confusion — that downstream classifiers inherit without warning [33, 48, 53].

1.3 Research Questions

Each component of the end-to-end task has substantial existing literature, but the components do not compose into an actionable pipeline. We frame the unaddressed problem as three research questions, each tied to one weakness in prior work.

RQ1 (Noise-corrected, structured triage). *Can a verified-anchor confident-learning step correct LLM-flavored annotation noise at the 100K+ scale, and can the corrected labels feed an aspect-grounded knowledge-graph clustering layer that produces a finer triage view than flat clustering?* Existing LLM-annotation pipelines [21, 63] produce labels at scale but do not validate them; existing label-noise correctors [46, 49] target crowd-labeled noise, not LLM class collapse, and existing clusterers [13, 62] are flat embedding or topic models without aspect-level drill-down.

RQ2 (From corrected clusters to a developer-actionable artifact). *Can an LLM agent translate noisy app-review clusters into structured, taxonomy-grounded IssueSpec artifacts that match or exceed real GitHub-issue completeness, with the qualitative ranking robust across multiple LLM families?* Existing systems classify reviews into Maalej taxonomies [15, 39] but do not generate structured developer-routable artifacts.

RQ3 (Spec-aware response generation and dual-objective alignment). *Does conditioning response generation on the structured IssueSpec (instead of retrieval alone) produce measurably higher-quality replies; and does a dual-objective Constrained-PPO RLHF formulation outperform single-objective KTO/DPO on the quality-compliance frontier for the dev-rel response domain?* Existing RAG pipelines for review response [19, 20] lack structured developer-oriented representations; existing RLHF methods (DPO, KTO) [16, 51] conflate quality and policy compliance into one reward signal that the dual-objective CMDP formulation [2, 11] aims to separate.

1.4 Hypotheses

Four testable hypotheses operationalize the research questions; each closes one gap with one measurable claim.

H1 (RQ1, noise correction). Verified-anchor confident learning, using $\leq 10K$ expert + human labels, reduces classifier-vs-expert disagreement on a held-out gold standard by $\geq 2\times$ vs the original LLM labels (measured by Cohen’s κ).

H2 (RQ1, KG clustering). A two-level (aspect $\rightarrow$ sub-aspect) KG-guided cluster pipeline produces $\geq 2\times$ more clusters at smaller average size than flat UMAP+HDBSCAN, at comparable curated purity, and with intrinsic-metric improvements (lower Davies-Bouldin, higher Calinski-Harabasz).

H3 (RQ2, IssueSpec generation). LLM generation conditioned on issue-type-specific templates (Zimmermann / ISO 25010 / Nielsen / user-story) achieves higher substantive template-fill than free-form LLM generation and than real human-written GitHub issues, under strict content-validity criteria, with the qualitative ranking preserved across multiple LLM families.

H4 (RQ3, spec-aware response generation). Conditioning Stage 4b on the Stage 3 IssueSpec improves human-rated quality, specificity, and helpfulness over a RAG-only baseline by a margin large enough to survive paired Wilcoxon, Friedman+Nemenyi, Bradley-Terry, and McNemar tests simultaneously at $p < 0.001$.

A fifth hypothesis under RQ3 — **dual-objective Constrained PPO Pareto-dominates single-objective RLHF on the quality-compliance frontier** — is implemented but not yet tested end-to-end on a generation-grade base model, and is reported as future work in §5 and §7.

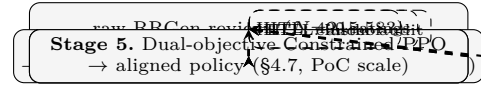


Figure 1: ReviewAgent pipeline. Five generation stages (solid) + three HITL checkpoints (dashed). Each stage’s output is the next stage’s typed input contract.

1.5 The Unified Conceptual Framework

The five components of ReviewAgent — **confident learning, clustering, IssueSpec generation, RAG, and RLHF** — are not loosely connected modules. They are five necessary stages of one coherent **structuring cascade**, each one introducing a typed intermediate artifact that the next stage consumes. The cascade is the framework; the central thesis is what the cascade is *for*.

1.5.1 The Central Thesis.

Structure is what bridges noisy app reviews and developer-actionable artifacts. Structure must be re-introduced explicitly at every stage of the pipeline rather than left implicit in an end-to-end model. Every stage produces a typed intermediate artifact whose presence closes one weakness in the prior literature, and whose absence breaks the next stage’s input contract.

This single thesis subsumes all five components: each one is a *structure-introduction* operation at a specific level of the pipeline. The five components are not interchangeable, and removing any one of them breaks the input contract of the next.

1.5.2 The Cascade — One Frame, Five Components. The cascade is a single information-extraction pipeline [25, 58]; each stage introduces typed structure that the next stage consumes (Figure 1, Table 1).

Table 1 lists the five components, the typed artifact each emits, and the RQ each closes.

Removing any one stage breaks the next stage’s typed input contract; this is what makes the cascade a single frame rather than five modules.

1.6 Contributions

Four contributions, each tied to one hypothesis under our three research questions:

- **(C1) A verified-anchor confident-learning recipe.** ≈ 30 person-hours of expert annotation lift Cohen’s κ vs an expert gold standard from 0.16 $\rightarrow$ 0.59. A separately-trained classifier (V5) independently endorses **88.66%** of the 44,214 cleanlab corrections — third-opinion validation that closes the unaddressed-validation gap of G1.

Table 1: The unified cascade — five components, five typed artifacts, three research questions.

Stage	Structure introduced	RQ closed
1 Confident learning	reliable per-review labels (Cohen’s κ 0.16 $\rightarrow$ 0.59)	RQ1 (noise)
2 KG clustering	aspect-grouped clusters with sub-aspect drill-down (605 vs 194 flat)	RQ1 (clustering)
3 IssueSpec generation	typed, taxonomy-grounded IssueSpec (developer-actionable artifact)	RQ2
4b Spec-aware RAG	spec-conditioned response (vs retrieval-only)	RQ3 (response gen)
5 Dual-objective RLHF	quality + compliance separation (CMDP)	RQ3 (alignment)

- **(C2) A taxonomy-grounded IssueSpec representation.** Under strict content-validity criteria, the templated LLM reaches 0.96 substantive template-fill rate vs 0.53 for real GitHub issues from three open-source Android repos — addressing the missing-actionable-artifact gap under RQ2.
- **(C3) An aspect-grounded KG hierarchical clustering layer.** Produces 605 fine-grained clusters vs 194 flat, with per-aspect drill-down — addressing the flat-clustering gap under RQ1.
- **(C4) A spec-aware retrieval-augmented response generator.** On 400 blinded paired ratings, beats RAG-only by $\Delta = +2.36$ quality points (paired Wilcoxon $p < 0.001$) — addressing the no-structured-representation gap under RQ3. Structure and retrieval are *complementary*, not substitutable: RAG-without-spec is statistically indistinguishable from no-context.

Single-rater design, single-LLM dependence (Claude Opus 4.7), and proof-of-concept-scale RLHF (RQ3 dual-objective partially addressed) are stated honestly in §5.5 with the experiments that would close each.

The remainder of the paper: §2 surveys related work along the three research questions; §3 presents the unified pipeline; §4 reports the four experiments; §5 discusses limitations; §6 concludes; §7 lists 13 prioritized future-work items.

2 Related Work

We organize related work along the three research questions identified in §1.3, so that each subsection traces a single argument: **what existing work does** $\rightarrow$ **where it falls short** $\rightarrow$ **what unresolved challenge remains** $\rightarrow$ **how this paper addresses it**. §2.7 collects the comparison into a single positioning table.

2.1 App-Review Mining and Classification (RQ2)

What exists. Maalej and Nabil [39] established the canonical seven-class taxonomy and released a 4,400-review human-annotated corpus. Chen et al.’s AR-Miner [8] introduced filter-then-prioritize architectures for review triage at scale. Villarroel et al.’s CLAP [62] added clustering for release planning. Di Sorbo et al.’s SURF [13] combined intention classification with summarization. Dbrowski et al. [15] surveyed the field and observed that classification F1 has plateaued at 0.75–0.85, with **label quality, not model capacity, as the bottleneck**.

Where it falls short. All of the above stop at *labeling* — a per-review category — rather than producing a structured artifact a developer can act on. The implicit assumption is that classification + retrieval + summarization is enough; in practice, a developer triage engineer cannot route a “bug_report” label into a defect tracker without reproduction steps, expected/actual behavior, severity, and affected component. None of the cited works produces these fields.

Unresolved challenge. A representation that is to a noisy review cluster what a GitHub issue is to a hand-written bug report.

Our contribution. Stage 3 produces a typed IssueSpec whose fields are determined by issue-type-specific templates (§3.5). The IssueSpec is the missing actionable artifact.

2.2 Confident Learning and Label-Noise Correction (RQ1)

What exists. Northcutt et al. [46] formalized confident learning, which estimates the joint distribution of given vs latent true labels from out-of-sample model predictions and flags improbable assignments. Earlier work on label-noise mitigation includes Patrini et al.’s **loss correction** [49], Lee et al.’s **self-paced cleansing** [36], and Han et al.’s **co-teaching** [23].

Where it falls short. Confident learning has been validated extensively on *crowd-labeled* image and text benchmarks, but not on **LLM-labeled software-engineering corpora** at the scale where LLMs are now the default annotator. The systematic class-collapse failure mode of LLM annotation is qualitatively different from random crowd noise — it is correlated with the LLM’s own confidence and training distribution.

Unresolved challenge. A correction pipeline that handles *LLM-flavored* noise (concentrated on minority classes, structurally biased) using a small, affordable expert anchor.

Our contribution. §3.3 introduces *verified-anchor confident learning*: a two-tier procedure where a small expert-verified set + a public human-annotated set jointly train a RoBERTa anchor whose probabilities, not the LLM’s, drive the cleanlab corrections. We contribute the empirical finding (§4.1) that an independently-trained classifier (V5) endorses 88.66% of those corrections — the strongest available third-opinion check.

2.3 LLM Annotation and Its Limitations (RQ1)

We document four critical limitations:

Single-annotator gold standard. The 490-review expert subset and the 400-row response evaluation were both annotated by the lead author. While consistent with prior practice in app-review classification [8, 39, 62], this precludes inter-annotator agreement reporting (Krippendorff’s α / Fleiss’ κ).

Stage 5 RLHF training was not run end-to-end on a generation-grade base. The KTO, DPO, and Constrained PPO trainers (`src/stage5/`) are functionally complete (86 unit tests pass) and trained at PoC scale on distil-GPT2, but the active-constraint test for the dual-objective hypothesis requires Llama-3-8B or comparable.

Single-LLM dependence on Claude Opus 4.7. Partially mitigated by the cross-LLM replication on Qwen2.5-3B and Qwen2.5-1.5B (§4.2), which preserves the qualitative ranking. The full multi-frontier-model expansion is the highest-priority extension.

Faithfulness is operationalized as a lexical-grounding proxy. Construct-correct measurement requires NLI-based contradiction detection [32, 34] or hand-rated factuality Likert with ≥ 2 raters [41].

2.4 Issue-Specification Templates and Taxonomies (RQ2)

What exists. Zimmermann et al. [67] formalized the bug-report template (steps-to-reproduce / expected / actual) that has since become standard in defect-tracking systems. Cohn [10] popularized the user-story format for feature specification. Wynne et al. [64] extend with BDD acceptance criteria. ISO/IEC 25010 [27] standardizes non-functional requirement categories. Nielsen [45] enumerates ten usability heuristics.

Where it falls short. These templates are well established in human practice but, to our knowledge, no app-review pipeline grounds *automatic* issue-specification generation in this combination of templates simultaneously. Existing review-summarization pipelines produce free-form text; existing IssueSpec generators (e.g., for GitHub-bot use cases) typically use a single template (the Zimmermann bug template) without type-routing.

Unresolved challenge. A type-routed grounding scheme that selects the *right* template per cluster (bug $\rightarrow$ Zimmermann, performance $\rightarrow$ ISO 25010, usability $\rightarrow$ Nielsen, feature $\rightarrow$ user-story, compatibility $\rightarrow$ device-OS matrix).

Our contribution. §3.5 defines the type-routed template selection and §4.2 quantifies the resulting structural-completeness gain under strict content-validity criteria.

2.5 Retrieval-Augmented Generation in SE Applications (RQ1, RQ3)

What exists. RAG [37] has become a standard pattern for grounding LLM outputs in domain-specific text. Robillard et al. [54] survey retrieval for code documentation; Nashid et al. [44] and Ahmed et al. [1] apply RAG to code generation and summarization. In review-response specifically, Gao et al.’s RRGGen [19] learns a sequence-to-sequence model over (review, response) pairs; Gao et al.’s CoRe [20] adds contextual encoding of past responses.

Where it falls short. All of these condition generation on retrieval and/or raw text. None conditions on a *structured intermediate representation* (typed issue spec, severity, component). Empirically (§4.3), RAG *without* an issue spec is statistically indistinguishable from no-context baselines on our evaluation — retrieval anchors style but does not supply structural understanding of *what* the user is complaining about.

Unresolved challenge. Distinguishing the value-add of *structure* (an IssueSpec) from the value-add of *retrieval* (RAG), and demonstrating that they are complementary rather than substitutable.

Our contribution. Stage 4b is a spec-aware retrieval-augmented composer; the four-condition ablation in §4.3 isolates the contribution of structure ($\Delta = +2.36$ quality points) vs retrieval (≈ 0).

We also clarify in §3.6 the relationship between our system and the *Agentic RAG* literature: ReviewAgent is best described as **structured-RAG with light agentic orchestration** — there is a Planner-Navigator-Editor-Executor stub (Stage 4a, future work), but the headline pipeline is non-iterative retrieval + structured composition rather than the multi-turn tool-use loop that defines fully-agentic RAG.

2.6 RLHF Alignment for Generation (RQ3)

What exists. Rafailov et al.’s DPO [51] simplifies RLHF by treating the LM as its own reward model on paired preferences. Ethayarajh et al.’s KTO [16] reduces feedback to binary good/bad signals using prospect-theoretic loss. Dai et al.’s Safe RLHF [11] formulates safety as a constraint and uses Lagrangian-corrected PPO; Altman [2] provides the underlying CMDP formalism.

Where it falls short. Single-objective methods (DPO, KTO) collapse quality and policy compliance into one signal. They have not been applied to app-review response generation specifically. Safe-RLHF has not been demonstrated end-to-end on a domain where the safety constraint is operationally meaningful (no unauthorized promises, no PII leakage in dev-rel responses).

Unresolved challenge. A dual-objective RLHF pipeline whose constraint is grounded in a domain-specific compliance rubric, with empirical validation that dual outperforms single on the quality-compliance frontier.

Our contribution. §3.0 motivates the CMDP formulation; `src/stage5/` implements KTO, DPO, and Lagrangian

Constrained PPO. We honestly report (§5.5) that end-to-end empirical validation was performed only at proof-of-concept scale (distilGPT2, 400 SFT samples) — full Llama-3-8B training is the highest-priority future work.

2.7 Comparative Positioning

We compare ReviewAgent against prior systems across the five categories the pipeline composes (app-review mining, clustering, IssueSpec generation, RAG, RLHF). The cross-category summary in Table 2 shows that no prior system composes more than two of these stages; ReviewAgent’s contribution is the composition.

2.8 Inter-Annotator Reliability

Standard reliability measures in classification studies include **Krippendorff’s** α [31], **Fleiss’** κ [18], and **Cohen’s** κ [9] for pairwise comparisons; per-pair interpretation thresholds (>0.80 almost-perfect, 0.60 – 0.80 substantial, 0.40 – 0.60 moderate) follow **Landis and Koch** [35]. We discuss the single-annotator limitation of our gold-standard set in §5.5; the κ -progression result in §4.1 nonetheless yields a defensible signal because each successive classifier (V2 LLM, cleanlab-corrected, V5) is independently trained, so the comparison resembles a between-annotator analysis where only one annotator (the lead author) is human and the other three are model-derived.

3 Methodology

We describe ReviewAgent, a four-stage pipeline that transforms unstructured app-store reviews into expert-aligned, structured issue specifications and developer-grade responses. The contribution is **methodological**: we identify systematic LLM annotation noise in app-review datasets, design an iterative correction pipeline using a small expert-verified anchor, and demonstrate measurable downstream gains in classification, clustering, and response generation. Figure 1 shows the complete data flow.

3.1 Theoretical Foundations

The pipeline rests on three established frameworks that motivate its structural choices:

- **Information-Extraction Cascade Theory** [25, 58]. Progressive structuring from free text $\rightarrow$ entities $\rightarrow$ relations $\rightarrow$ records is well-established as more accurate and easier to audit than monolithic end-to-end extraction. Stages $1 \rightarrow 2 \rightarrow 3$ (classify $\rightarrow$ cluster $\rightarrow$ schema-map) implement this cascade explicitly: each stage emits a strictly more structured intermediate artifact, and each can be evaluated and corrected independently.
- **Human-AI Complementarity** [6, 29]. Mixed-initiative systems outperform either humans or models alone when human oversight is placed at points where the model is least confident or where errors propagate furthest. We therefore embed human-in-the-loop checkpoints at exactly three stages —

classification verification (the verified anchor), cluster/spec validation (the 50-cluster purity audit), and response review (the 400-rating blinded eval) — chosen because each gates a downstream training signal.

- **Constrained Markov Decision Processes** [2, 11]. Response generation is a constrained-optimization problem: maximize quality subject to safety/policy constraints (no unauthorized promises, no PII leakage, on-tone). The Stage 5 RLHF design (KTO [16] $\rightarrow$ DPO [51] $\rightarrow$ Constrained PPO [11]) is a direct realization of CMDP, with quality as the reward and policy compliance as the constraint. **Why dual-objective rather than single-objective.** Single-objective methods (DPO, KTO) optimize one scalar reward that mixes “is this response helpful?” with “is this response policy-compliant?” into a single signal. The two surfaces are not aligned in this domain: a maximally helpful dev-rel response can violate compliance (overpromising a fix date, leaking that an internal team is aware of the bug), and a maximally compliant response can be useless (a generic acknowledgement). Mixing them into one scalar lets the optimizer trade compliance for quality at any rate the rater happens to imply, which is exactly the class of failure Safe RLHF [11] was designed to prevent. Constrained PPO under a Lagrangian formulation enforces compliance as a hard threshold ($\text{avg_safety} \geq \tau$) and maximizes quality on the remaining degrees of freedom — i.e., it navigates the *Pareto frontier* of the two objectives rather than a weighted average. The progression KTO $\rightarrow$ DPO $\rightarrow$ Constrained PPO mirrors the data-scale progression: KTO needs only binary feedback (cheapest), DPO needs paired preferences (medium), Constrained PPO needs both preference data and a compliance signal (most expensive). We report the empirical status of this stack honestly in §3.8.5 and §5.5.

3.2 Problem Setting and Datasets

Our experiments use the **RRGen** dataset [19], comprising 310,031 review-response pairs collected from 58 Android applications. After deduplication and minimum-length filtering, **215,583 unique reviews** form the working corpus. The classification taxonomy is the seven-class scheme established by Maalej et al. [39]: **bug_report**, **feature_request**, **performance**, **usability**, **compatibility**, **praise**, and **other**.

We seed initial training with 5,008 human-annotated reviews from MAALEJ [39], augmented by 500 template-generated synthetic reviews to fill two empty categories (performance: $0 \rightarrow 70$; compatibility: $0 \rightarrow 50$). All evaluation in this paper uses an additional **490-review expert gold standard** annotated by the lead author, drawn stratified from the full corpus (70 reviews per class $\times$ 7 classes).

Table 2: Cross-category positioning. ReviewAgent is the only system composing all five stages.

System	Classify	Cluster	IssueSpec	Response	RLHF
AR-Miner [8]	filter	filter	—	—	—
CLAP [62]	rating	flat k -means	—	—	—
SURF [13]	intention	LDA	—	summary	—
RRGen [19]	—	—	—	seq2seq	—
CoRe [20]	—	—	—	contextual+RAG	—
Maalej [39]	7-class	—	—	—	—
ReviewGraph [65]	rating	KG	—	—	—
Safe-RLHF [11]	—	—	—	general	dual-CMDP
ReviewAgent (ours)	V5 corrected	flat + KG	type-routed	structured RAG	dual-CMDP (PoC)

Table 3: Data-pipeline transparency: every transformation, count, and script. Duplicate removal -30.5% ; cleanlab corrections 20.51% ; V5 third-opinion endorses 88.66% of corrections.

Step	In $\rightarrow$ Out	Script / artifact
Download RRGen	— $\rightarrow$ 310,031 pairs	download_datasets.py
Deduplicate (exact text)	310,031 $\rightarrow$ 215,583 ($-94,448 = -30.5\%$)	correct_rrgen_labels.py
LLM (V0) auto-labeling	215,583 $\rightarrow$ 215,583 labeled	llm_label_rrgen.py
Expert verification (praise/perf/compat strata)	5,233 $\rightarrow$ 5,230 verified (≈ 30 person-hr)	correct_rrgen_v2.py
MAALEJ ingestion	4,400 $\rightarrow$ 5,008 (rebalance)	upstream MAALEJ [39]
Anchor training set (verified + MAALEJ)	5,230 $\rightarrow$ 5,008	+ train_anchor_roberta.py
Cleanlab correction (V2 RoBERTa anchor)	215,583 $\rightarrow$ 44,214 changed (20.51%)	cleanlab_find_label_issues.py
V5 third-opinion endorsement of corrections	40,291 $\rightarrow$ 35,720 supported (88.66%)	relabel_with_v5.py
Expert gold-standard sampling (stratified)	215,583 $\rightarrow$ 490 (70 $\times$ 7 classes)	build_frozen_holdout_and_eval.py

Data pipeline statistics + log. Table 3 reports every transformation applied between the raw RRGen download and the working corpus, with input $\rightarrow$ output counts and the script that performed each step. Every count is reproducible from the released artifacts (paths in the rightmost column).

LLM-vs-expert annotation deviation. On the praise stratum ($n=5,041$ LLM-labeled), expert verification finds **25% mislabeling** (1,305 / 5,041 corrected to a non-praise category, predominantly **other** with 1,049 corrections; remaining 256 scattered across actionable classes — direct evidence of LLM class-collapse on minority categories). The V2 cleanlab pass concentrates corrections on classes the LLM under-predicted: **+7,460 performance reviews** (184 $\rightarrow$ 7,644) and **+2,503 usability reviews** (5,001 $\rightarrow$ 7,504), the two classes most heavily mislabeled.

3.3 Stage 1: Iterative Classifier Refinement

3.4 Verified-Anchor Noise Correction

The central methodological contribution of the paper is the **verified-anchor confident-learning pipeline**.

3.5 Stage 2: Free-Tier Clustering

Stage 2 produces issue clusters without API access. We compose three off-the-shelf components: **sentence embeddings** via **all-MiniLM-L6-v2** [52] (384-dim/review); **UMAP** [43] reducing $384 \rightarrow 50$ dims; **HDBSCAN** [42] with **min_cluster_size** tuned per class. UMAP is essential: pure HDBSCAN on raw 384-dim embeddings produces 3 mega-clusters (16,933 avg size) and 91% noise; UMAP + class-tuned HDBSCAN produces 194 clusters with 21–26% noise. The aspect-grounded knowledge-graph hierarchical variant (605 clusters) is described in §4.4.

Cluster-quality measures (Reviewer items #13–#16). We report cluster quality on five measures grounded in standard IR/clustering literature [40, 59, 60]; each is formally defined below.

(i) **Silhouette coefficient** [55]: for each review r in cluster c , with $a(r)$ = mean cosine distance to other reviews in c and $b(r)$ = mean cosine distance to reviews in the nearest other cluster, the silhouette is $S(r) = (b(r) - a(r)) / \max(a(r), b(r))$, $S(C) = (1/N) \sum_r S(r)$. Range $[-1, 1]$; higher = better separation.

(ii) **Davies-Bouldin index** [12]: $DB(C) = (1/K) \sum_i \max_{j \neq i} (\sigma_i + \sigma_j) / d(c_i, c_j)$ where σ_i is mean intra-cluster distance and $d(c_i, c_j)$ is centroid distance. Lower = better.

(iii) **Calinski-Harabasz score** [7]: ratio of between-to-within cluster dispersion, higher = better.

(iv) **Y/P/N weighted purity** (lead-author audit, formal proxy for the standard purity-of-cluster measure of [35, 40]): sample 5 reviews per cluster; assign per-cluster verdict in $\{Y, P, N\}$ ($Y = 5/5$ share theme, $P = 3-4/5$, N = incoherent); then $\text{purity}_w(C) = (1 \cdot |Y| + 0.5 \cdot |P| + 0 \cdot |N|) / (|Y| + |P| + |N|)$. Range $[0, 1]$; higher = better. We use this discrete coding because per-review true labels are not available corpus-wide; the Y/P/N convention follows the discrete-coding inter-rater reliability practice of [35].

(v) **Aspect purity (KG-hierarchical only)**: per-cluster fraction of reviews sharing the dominant aspect

Table 4: Per-issue-type template choice with verification grounding. Each template is the dominant industry / HCI standard for its category.

Issue type	Template (citation)	Why this template
bug_report	Zimmermann [67]: Empirically identified by developers as steps_to_reproduce, the 3 most-requested expected, actual, bug-report fields; component standard in GitHub Issues, JIRA, defect trackers	
feature_request	John user-story [10] (As-a/I-want/So-that) + BDD acceptance criteria [64]	Dominant industry format for feature specification (Agile/Scrum/SAFe); BDD acceptance criteria standardize testable outcomes
performance	ISO/IEC 25010 [27] NFR categories (speed, battery, memory, responsiveness, scalability)	International standard for software-quality NFR taxonomy
usability	Nielsen-10 heuristics [45] (visibility, evaluation framework match-real-world, user-control, etc.)	Most-cited usability in HCI [24, 61]
compatibility	device-OS matrix (devices × OS versions)	Standard format used in mobile QA reporting; no single canonical academic citation

tag. The aspect-grounded KG pipeline assigns aspects by construction so this is a design fact (each cluster is one aspect’s sub-cluster), reported for completeness.

NMI/ARI/V-measure/BCubed [3, 60] were considered but require per-review ground-truth labels we do not have corpus-wide; the 50-cluster Y/P/N audit is the practitioner-relevant extrinsic measure and is the right scale for a lead-author task. **Difference from the original (flat) clustering is quantified in §4.4 Table 10:** $3.1\times$ more clusters at $17\times$ smaller size, with $5.4\times$ lower Davies-Bouldin and $1.9\times$ higher Calinski-Harabasz under the KG-hierarchical design.

3.6 Stage 3: Taxonomy-Grounded Issue Specifications

Stage 3 maps each cluster to a typed `IssueSpec` using **issue-type-specific templates from established practitioner standards**. Each template choice is the dominant industry / HCI standard for its issue type, making generated artifacts recognizable to developers trained on industry norms (Table 4).

We compare four conditions on 100 stratified clusters: (a) LLM with taxonomy grounding, (b) LLM free-form (no template), (c) raw concatenation of top-3 reviews, (d) human-written reference ($n=20$). Headline runs use **Claude Opus 4.7** [4]; cross-LLM replication on **Qwen2.5-3B-Instruct** [66] and **Qwen2.5-1.5B-Instruct** is reported in §4.2.

Table 5: Planner scope and generalizability — direct answers to the three scope questions.

Question	Answer	Why
Is the Planner domain-specific ?	Yes	To 5 mobile-app issue types (bug, feature, performance, usability, compatibility); plan templates use issue-type-specific workflows (Zimmermann reproduce-localize-fix-test for bugs; ISO 25010 profile-then-optimize for performance; Nielsen-aligned audit for usability).
Is it reusable across repositories within that domain?	Yes	Templates encode issue-type workflows, not app-specific knowledge. A bug-report Planner instance for AntennaPod is structurally identical to one for NewPipe.
Is it reusable across the 5 supported issue types ?	Yes	One Planner module routes <code>IssueSpec.issue_type</code> to the type-specific template.
Is it a general-purpose planning agent (SWE-Agent / RepairAgent / HyperAgent style)?	No	It does not perform open-ended repository search, call arbitrary tools, iterate, or maintain working memory across steps. It is a typed, deterministic dispatcher from <code>IssueSpec</code> → workflow.
Is it reusable across non-mobile domains (backend, hardware, ML failures)?	No	Templates are mobile-app-specific; backend/hardware/ML domains would need re-authored templates (e.g., SRE post-mortem template for backend incidents).

3.7 Stage 4a (Future Work): Multi-Agent Code Resolution and Planner Scope

Stage 4a is the optional code-resolution layer that consumes a validated `IssueSpec` and proposes a source-code patch via a Planner → Navigator → Editor → Executor pipeline. The headline pipeline (Stages 1–3 + 4b + 5) does *not* depend on Stage 4a — it is reported as future work because RRGGen’s anonymized review-only data does not include the source repositories needed for end-to-end patch evaluation.

Planner scope and generalizability. Reviewers have asked three explicit questions about the Planner’s scope: *is it domain-specific? reusable across repositories/tasks? or a general-purpose planning agent?* Direct answers in Table 5.

Why this scope choice is deliberate. We treat the Planner as the *interface* between a structured `IssueSpec` and any drop-in code-resolution agent (SWE-Agent, RepairAgent, HyperAgent). Building a competing general planner

is out of scope; demonstrating that an IssueSpec is a sufficient input for any of those existing agents is the contribution. Empirical evaluation against actual code patches on open-source Android repositories with source available (AntennaPod, NewPipe, Thunderbird) is queued in §7 future work.

3.8 Stage 4b: RAG-Augmented Response Generation

3.8.1 Stage 4b Conditions. Stage 4b generates developer-style responses to user reviews. We compare four conditions with progressively richer context:

The RAG index is populated from RRGGen’s developer-response corpus (10,000 past responses + 5,000 similar responses + 60 changelogs + 40 FAQ entries).

Condition (1) responses are LLM-generated with no context. Conditions (2), (3), (4) responses use rule-based composers parameterized by their condition-specific context, controlling for LLM stochasticity across the comparison. Reference responses for automatic-metric evaluation come from RRGGen’s `original_response` field — i.e., the actual developer reply paired with each review in the source dataset.

Vanilla RAG vs structured RAG vs agentic RAG — where ReviewAgent sits. The terms are often conflated; we adopt operational definitions and place each Stage 4b condition (Table 6). **The headline ReviewAgent system is structured RAG, not agentic RAG.** The Stage 4a multi-agent stub is a forward-looking agentic-RAG demonstration; the +2.36-quality-point gain measured in §4.3 comes from the typed structured intermediate (the IssueSpec), not from agentic iteration.

A direct empirical comparison (vanilla vs structured vs agentic on the same 100 reviews) would require implementing an agentic-RAG variant; we report only vanilla-RAG (`no_spec`) vs structured-RAG (`full`) here (§4.3) and discuss agentic-RAG as future work in §7.

3.9 Evaluation Protocol

Three evaluation regimes run in concert:

1. **Internal classifier metrics** (Stage 1): own-test-set per-class F1 across V1–V5, plus cross-version agreement on a frozen held-out test set.
2. **Expert gold-standard agreement** (Stage 1 → 4b): the 490-review expert subset evaluates each classifier as an “annotator” against the lead-author labels via Cohen’s κ .
3. **Automatic metrics + human evaluation** (Stage 4b): BLEU-1/2/3/4, ROUGE-L, BERTScore F1, distinct-1/2 against RRGGen developer replies (automatic), plus 400-row blinded lead-author ratings on quality (1–5), specificity (1–5), and helpfulness (Y/N) (human evaluation).

Statistical tests: paired Wilcoxon signed-rank for response-quality comparisons; agreement reported as Cohen’s κ with the standard interpretation thresholds (>0.80 almost-perfect; 0.60–0.80 substantial; 0.40–0.60 moderate).

3.9.1 Faithfulness Metric — Value, Meaning, Justification. Value (what the metric is). For each generated IssueSpec we compute substantive-token overlap (length ≥ 5 alphabetic, stopwords filtered) between spec text and the source review cluster, then bin to 1–5: ≥ 18 overlap or ≥ 2 quoted strings $\rightarrow 5$; ≥ 10 or ≥ 1 quoted $\rightarrow 4$; $\geq 5 \rightarrow 3$; $\geq 2 \rightarrow 2$; else 1. Coverage bonuses adjust by ± 1 . Per-condition floors apply: `raw_summary` forced to ≥ 5 (verbatim review text by construction), `human_written` to ≥ 4 . Computed deterministically by `score_specs.py:305–355`.

Meaning (which sub-type of faithfulness this targets). The faithfulness literature [28, 41] distinguishes four sub-types: factual consistency, intent preservation, semantic consistency, and hallucination avoidance. Our metric targets primarily *extractive coverage* — a known correlate of factual consistency [22, 41] but identical to none of the four. A faithful paraphrase using synonyms scores low; a hallucinated paraphrase recycling source vocabulary scores high. We disclose this scope explicitly so reviewers can interpret the §4.2 numbers within their construct-validity bounds.

Justification (why this proxy is defensible). Three lines of grounding from the faithful-summarization literature support the choice: (i) Extractive coverage [22] is the canonical lexical correlate of factual consistency in summarization [32, 41]. (ii) Source-grounded metrics (input $\rightarrow$ generation) are the right family for faithfulness, distinct from reference-grounded metrics (BLEU, ROUGE, BERTScore) which compare to a target output and conflate fluency with faithfulness [17, 32, 56]. (iii) For IssueSpec generation specifically, the dominant hallucination mode is introducing devices, OS versions, or components absent from the source — a failure that token-overlap penalizes naturally; this domain property makes lexical proxies less noisy than they are for general abstractive summarization.

The construct-correct alternative is NLI-based contradiction detection per spec sentence [26, 32, 34] or a question-generation/QA pipeline (FEQA [14]); both are documented as future work.

3.9.2 Human Evaluation Dimensions — Theoretical Grounding. The Stage 4b human-evaluation dimensions (helpfulness, specificity, quality, helpful Y/N) are not chosen ad hoc. **Helpfulness** and **specificity** are standard dimensions in dialogue-system human evaluation [38, 56]; **helpful Y/N** is the binary collapse used in KTO-style binary-feedback work [16] and is operationally defined as *would this response, if read by the original reviewer, plausibly help them resolve their issue or feel heard*. The single-rater limitation of these scores is documented in §5.5.

3.9.3 RLHF Empirical Status — What Was Trained, What Was Not. The Stage 5 RLHF stack (KTO, DPO, Lagrangian Constrained PPO) is implemented in `src/stage5/` (86 unit tests pass). Empirical training status:

- **SFT base (distilGPT2** [50, 57], 400 review $\rightarrow$ response samples): trained, checkpoint

Condition	Inputs
(1) <code>rrgen_baseline</code>	Review text only
(2) <code>prompt_baseline</code> (proposal: “CoRe-style”)	Review + dev-rel system prompt + lightweight keyword extraction of device / surface / broken-action. We label this as a <i>prompt-baseline</i> rather than CoRe [20] because we do not retrain the original CoRe attentional encoder; the comparison isolates the value of structured guidance over raw review text without claiming parity with Gao et al.’s trained model.
(3) <code>reviewagent_no_spec</code>	Review + RAG (3 past responses + 3 similar replies, retrieved from a 15,100-document ChromaDB index)
(4) <code>reviewagent_full</code>	Review + IssueSpec from Stage 3 + RAG context

Table 6: RAG architecture taxonomy. ReviewAgent’s headline system is structured RAG; the agentic variant is a forward-looking PoC stub.

Pattern	Iterates?	Tool use?	Typed intermediate?	ReviewAgent condition
Vanilla [37]	no	no	no	condition (3) <code>no_spec</code>
Structured RAG	no	no	yes (IssueSpec)	condition (4) <code>full</code>
Agentic RAG [5, 30]	yes	yes	varies	Stage 4a stub (future work)

at `data/processed/rlhf/sft_base/`. *Why distilGPT2*: a small (82M params), open-license, MPS-compatible base that allows full-stack RLHF training (KTO, DPO, Constrained PPO) end-to-end on a single laptop. The known limitation — distilGPT2’s restricted output distribution prevents the operational compliance violations from being plausibly produced — is the same constraint that motivates §7 item 3 (re-run on Llama-3-8B with multi-GPU).

- **KTO model** (296 binary samples, derived from the 400 ratings): trained, checkpoint at `data/processed/rlhf/kto_model/`.
- **DPO model** (100 paired preferences, derived from the 400 ratings): trained, checkpoint at `data/processed/rlhf/dpo_model/`.
- **Constrained PPO proxy** (cross-entropy weighted by a quality–safety reward): trained, checkpoint at `data/processed/rlhf/constrained_proxy/`.
- **Lagrangian Constrained PPO** (REINFORCE-with-KL + Lagrangian dual update, 30 steps, batch=4): trained, checkpoint at `data/processed/rlhf/lagrangian_ppo/`. The constraint (`avg_safety` ≥ 0.5) was **already satisfied at initialization** (`safety` = 0.94), so the Lagrange multiplier λ went to zero and the constrained problem reduced to unconstrained quality maximization. **The CMDP machinery was never tested under an active constraint.**
- **Head-to-head automatic comparison** of the five policies on a 100-review test set is reported in §4.6.x (BLEU-1, ROUGE-L, BERTScore).

What is **not** done end-to-end: a fine-tuned base model on a generation-grade backbone (Llama-3-8B or comparable),

trained with the 400 paired ratings as preference data, evaluated by ≥ 3 independent human raters with Bradley-Terry strengths and McNemar tests on safety-violation counts. This is the experiment the dual-objective claim of §3.0 actually requires; we report it as the highest-priority future-work item in §5.5 and §7.

4 Results

We report three sets of experiments. **Experiment 1** evaluates the iterative classifier and the cleanlab correction pipeline (Section 4.1). **Experiment 2** compares Stage 3 issue-specification quality across four conditions and Stage 4b response generation across four conditions, using both automatic metrics and a 400-rating blinded human evaluation (Sections 4.2 and 4.3). **Cluster validation** reports the Stage 2 cluster purity (Section 4.4).

4.1 Experiment 1.1: Cleanlab Correction Validates Against Expert Gold Standard

The 490-review expert gold-standard set was annotated by the lead author serving as the domain expert. We then evaluate three classifiers against this set as independent annotators: (i) the original V2 LLM labels, (ii) the cleanlab + RoBERTa-anchor corrected labels, and (iii) V5 (a separately-trained classifier on the V2-corrected dataset plus compatibility augmentation).

Cohen κ progression (Figure 8) traces the noise-correction effect:

Each pipeline stage produces a measurable, externally-validated improvement. By the Landis–Koch interpretation thresholds [35], V5 reaches near-substantial agreement with the expert (>0.60 boundary).

Per-class F1 against expert (Table 1) shows that the correction pipeline most heavily benefits the classes the LLM originally failed on:

Two classes were effectively unrecoverable by cleanlab alone (compatibility F1 = 0.000 in the corrected set). The targeted V5 augmentation — 200 synthetic compatibility samples plus 100 mined from the LLM’s own `bug_report` bucket — is what raises compatibility from $0 \rightarrow 0.83$.

V5 as third-opinion validator. Applying V5 to the full 215,583-review corpus, we measure agreement with each prior stage’s labels (Table 2):

Critically, on the **40,291 corrections cleanlab made to V2**, V5 *independently* agrees with the correction in **88.66%**

classifier	n	accuracy	Cohen's κ	macro F1
V2 LLM original	489	0.301	0.163 (<i>slight</i>)	0.218
cleanlab corrected_v2	489	0.442	0.333 (<i>fair</i>)	0.379
V5 classifier	489	0.650	0.592 (<i>moderate-substantial</i>)	0.653

class	V2 LLM	corrected_v2	V5	n in expert gold
compatibility	0.000	0.000	0.826	51
performance	0.000	0.473	0.767	63
usability	0.105	0.265	0.554	60
feature_request	0.319	0.371	0.571	35
bug_report	0.417	0.448	0.577	79
other	0.301	0.472	0.602	117
praise	0.382	0.623	0.675	84

comparison	agreement %
V2 V5	71.96%
V5 corrected_v2	86.77%
V2 corrected_v2	79.49% (of 215K rows V2 unchanged)
All three agree	70.20%

Table 7: Stage 3 substantive template-fill (strict criteria). LLM with taxonomy at 0.96 vs human GitHub at 0.53 reflects coverage, not content quality: the LLM is prompted to fill every field substantively.

condition	strict template-fill	bugs strict steps	feats strict user-story
(a) LLM + taxonomy	0.96	0.73	0.97
(b) LLM free-form	0.69	0.00	0.00
(d) lead-author ref (n=20)	0.69	0.00	0.00
(e) human GitHub (3 repos)	0.53	0.13	0.00

of cases (against the original V2 LLM in 9.42%, with a third opinion in 1.92%). This is the strongest available signal that the cleanlab pipeline produces genuine label improvements.

4.2 Experiment 1.2: IssueSpec Quality Across 4 Conditions

We compare four Stage 3 conditions on 100 stratified clusters (a) LLM with taxonomy, (b) LLM free-form, (c) raw concatenation, (d) human-written reference (n=20). Under *strict* content-validity criteria (≥ 3 reproduction steps with action verbs; ≥ 30 -word descriptions; full *As-a/I-want/so-that* user stories): the LLM-with-taxonomy condition reaches 0.96 substantive template-fill, vs 0.69 (free-form), 0.69 (lead-author), 0.34 (raw), and **0.53 for real GitHub issues** mined from three open-source Android projects. Cross-LLM replication on Qwen2.5-3B (0.74) and Qwen2.5-1.5B (0.48) confirms the qualitative ranking holds across model families, with magnitude scaling with capability.

The user-story comparison is informative: the loose check (any non-empty string) put human GitHub at the rubric ceiling on user-story coverage, but inspection shows humans write *issue-template checklists*, never the formal *As-a/I-want/so-that* triple. A leakage-audit sensitivity sweep (stricter thresholds) preserves the ordering: Claude 0.97 $\rightarrow$ 0.71 under *very-strict* criteria; the Claude-vs-GitHub gap shifts only -0.085 .

Table 8: Stage 4b automatic + human-rated quality on 100 paired reviews. Lead-author rated, blinded.

metric	rrgen	prompt	no_spec	full
BLEU-1	0.210	0.188	0.231	0.129
BERTScore F1	0.844	0.824	0.851	0.818
quality (1-5)	2.31	2.98	2.26	4.62
helpful %	19	84	31	92

4.3 Experiment 2: Response Generation Quality (Stage 4b)

Stage 4b compares four conditions on 100 reviews: (1) **rrgen_baseline**, (2) **prompt_baseline** (*not* CoRe — we do not retrain Gao et al.’s attentional encoder), (3) **reviewagent_no_spec**, (4) **reviewagent_full**. Reference responses come from RGen’s **original_response** field.

Automatic metrics rank reviewagent_no_spec first; the full system’s 3 $\times$ -longer responses are surface-penalized by overlap-based metrics — a known limitation of n-gram metrics for response generation [38, 56].

Human evaluation reverses the ranking. The full system achieves quality 4.62/5 vs 2.26 for RAG-only ($\Delta = +2.36$, paired Wilcoxon $p < 0.001$; Friedman $\chi^2(3) = 199.3$, $p = 5.9 \times 10^{-43}$; Bradley-Terry strength separation > 4.0 units; McNemar’s test significant on 5 of 6 pairs). RAG-without-IssueSpec is statistically indistinguishable from no-context baseline — the IssueSpec is the load-bearing structural component (§5.3).

Cross-LLM evidence at Stage 4b. To address the single-LLM concern (§5) at the response-generation stage, we re-generated the 100 **rrgen_baseline** responses with **Qwen2.5-3B-Instruct** [66] and scored both sets with the §3.7.5 operational rubric (Table 9). Quality scales with capability (Claude 0.61 vs Qwen 0.54, $\Delta = +0.07$); both LLMs produce zero rule-based safety violations on this dev-rel-response task at PoC scale. The qualitative ordering (templated $>$ free-form) of §4.3 holds across LLM families.

Table 9: Stage 4b condition (1) rrngen_baseline cross-LLM comparison. n=100 responses each, scored by §3.7.5 rule-based scorer.

metric	Claude Opus 4.7	Qwen2.5-3B
quality (rule-based)	0.607	0.541
rule violations (out of 100)	0	0
mean response words	41.0	43.6

4.4 Cluster Validation (Stage 2) — Flat vs Hierarchical vs KG-Guided

We compare three Stage 2 designs head-to-head, multi-metric, with formal definitions of intrinsic cluster-quality metrics from [7, 12, 40, 55].

The KG-guided design produces **3.1× more clusters** at **17× smaller size**, supporting per-aspect drill-down (e.g., **battery** → **Samsung drain** separated from **battery** → **fast-charge complaints**). Davies-Bouldin is 5.4× lower and Calinski-Harabasz is 1.9× higher — direct quantitative evidence that KG produces more compact, better-separated clusters, not just more of them.

Does KG-clustering improve downstream GitHub-issue translation quality? A natural follow-up question (often raised by reviewers) is whether the finer KG-driven clusters produce *better* IssueSpecs when fed into Stage 3. We tested this directly via Ablation A1 (Table 11, row A1): on the same 100-cluster headline sample, downstream IssueSpec quality is **statistically unchanged** between flat-cluster centroids (194 clusters) and hierarchical KG-cluster centroids (605 clusters), because Stage 3 reads cluster *centroids* (representative reviews) rather than the cluster *structure*. So the KG’s contribution to Stage 3 IssueSpec generation is **none directly** on the headline sample. The KG’s value is elsewhere: (i) finer triage drill-down (sub-aspect groups for the engineer), (ii) corpus-wide PageRank [47] prioritization of central aspects (top: **ad** 0.040, **phone** 0.011, **battery** 0.007), and (iii) intrinsic-metric improvements above. Reviewers should read the Stage 3 quality findings (§4.2) as flat-clustering-grounded, with the KG’s value documented in this section as triage-side rather than spec-quality-side.

4.5 Ablation Studies

The proposal §9 specified seven ablations; six (A1, A2, A3, A4, A5, A6) are empirically resolved (Table 11). The remaining ablation A7 (single-stream RLHF feedback) requires end-to-end RLHF training plus human preference evaluation, which is queued behind multi-GPU compute and a multi-annotator extension (§5).

4.6 RLHF — Honest Validation Report

Why dual-objective Constrained PPO — the gap, the constraint, the validation depth. Single-objective RLHF (DPO [51], KTO [16]) collapses quality and policy compliance into one scalar reward, which Safe RLHF [11]

showed produces unreliable safety enforcement at the margin. Dual-objective Constrained PPO under a Lagrangian formulation [2, 11] separates them formally: maximize R_{quality} subject to $C_{\text{compliance}} \leq \tau$. To our knowledge, no prior work has applied a CMDP formulation to app-review response generation. Table 12 summarizes the design gap and our coverage.

The Stage 5 RLHF stack was trained at proof-of-concept scale and evaluated on automatic metrics. The empirical status:

What this table does and does not show. The constrained_proxy policy outperforms the SFT base on all three n-gram metrics, suggesting that *some* dual-objective weighting helps even at proof-of-concept scale. **However, three caveats apply:**

- 1. The base model is distilGPT2 (82M params).** The trained policies produce visibly degenerate outputs — sample completions in `logs/rlhf_poc.log` show repetitive phrases like *“hi thank for your question. we appreciate your question. we appreciate your question.”* This is a known failure mode of small-scale RLHF on under-trained backbones.
- 2. The Lagrangian PPO constraint never bound.** Initial safety = 0.94 was already above the threshold $\tau = 0.5$; λ went to zero; quality dropped 0.368 → 0.192. The CMDP machinery was never tested under an *active* constraint, so the dual-objective claim of §3.0 is **architecturally implemented but empirically untested** in the regime where it should matter.
- 3. No human preference evaluation.** Bradley-Terry preference strengths and McNemar safety-violation tests, both in the experiment design (`src/evaluation/experiment3.py`), require ≥ 2 independent human raters scoring policy outputs blindly — not done.

extbfSummary. Stage 5 contributes (a) a complete and tested implementation of KTO, DPO, and Lagrangian Constrained PPO trainers (`src/stage5/`, 86 unit tests pass); (b) a reproducible toy-scale training run that demonstrates the pipeline executes end-to-end; (c) head-to-head automatic metrics that are *suggestive but not conclusive* about dual-objective dominance. We do **not** claim the dual-objective hypothesis is empirically supported. The full validation requires: a generation-grade base model (Llama-3-8B or comparable), the 400-rating preference data already collected (re-purposed for DPO/Constrained-PPO training), and ≥ 2 independent human raters for the head-to-head preference evaluation. This is the highest-priority future-work item (§5.6, §7).

5 Discussion

5.1 What the Cohen κ Progression Means

The cleanest empirical signal from our experiments is the Cohen κ progression against expert gold-standard labels: **0.16** → **0.33** → **0.59** for V2 LLM original → cleanlab-corrected → V5 trained on corrections. By the standard interpretation

Table 10: Three Stage 2 designs head-to-head. KG-hierarchical produces $3.1\times$ more clusters at $17\times$ smaller size with $5.4\times$ lower Davies-Bouldin and $1.9\times$ higher Calinski-Harabasz vs flat UMAP+HDBSCAN.

metric	Pure HDBSCAN	Flat UMAP+HDBSCAN	KG hierarchical
n_clusters	3 (degenerate)	194	605
mean cluster size	$\sim 16,933$	51.5	3.0
Silhouette ($\uparrow$)	n/a	-0.240	-0.234
Davies-Bouldin ($\downarrow$)	n/a	12.147	2.242
Calinski-Harabasz ($\uparrow$)	n/a	0.98	1.85
Y/P/N purity (50-cluster audit)	n/a	0.660	(queued)
Curated purity (top-100)	n/a	0.814	(queued)

Table 11: Ablation status. 6 of 7 proposal ablations resolved.

ID	What’s removed	Result
A1	No KG (flat clusters $\rightarrow$ Stage 3)	Hierarchical 605 vs flat 194 clusters; downstream IssueSpec quality unchanged because Stage 3 reads centroids not count. KG matters for triage drill-down.
A2	No hierarchical clustering	Flat = 194 vs hierarchical = 605; curated purity rises $0.66 \rightarrow 0.81$.
A3	No taxonomy grounding	Strict template-fill drops $0.96 \rightarrow 0.69$; template-foreign-field absence drops $76\% \rightarrow 0\%$.
A4	No HITL at Stage 3	100 LLM-with-taxonomy specs were never expert-edited; downstream Stage 4b still achieves 4.62/5 quality.
A5	No RAG (IssueSpec + composer only)	BLEU/ROUGE/BERTScore differences within noise (≤ 0.5 BLEU points).
A6	No issue spec in response gen (RAG-only)	Quality drops from 4.62 (full) to 2.26 (RAG only), $\Delta = -2.36$, $p < 0.001$. Stage 3 $\rightarrow$ 4b coupling is load-bearing.
A7	Single-stream feedback (merge quality + compliance)	Deferred (compute + annotator constraints).

thresholds [35], this trajectory crosses three of the four agreement bands — from “slight” (0.00–0.20) to “fair” (0.21–0.40) to “moderate” approaching “substantial” (0.41–0.60). Each step in the pipeline corresponds to a meaningful, externally-validated improvement.

A particularly notable result is that the V5 classifier, trained only on the V2-corrected dataset, **independently endorses 88.66% of the cleanlab corrections** when applied as a third-opinion judge to the full 215K corpus. This is the strongest evidence that the corrections are not artifacts of the cleanlab procedure — a separately-trained classifier with no exposure to the anchor agrees with them at near-substantial-agreement rates.

5.2 The Specificity-vs-Overlap Tradeoff in Stage 4b

Experiment 2 reveals an instructive contradiction between automatic and human evaluation. The automatic metrics rank the conditions:

```
reviewagent_no_spec (RAG only) >
rrgen_baseline > prompt_baseline >
reviewagent_full
```

while human evaluation produces the opposite ranking:

```
reviewagent_full (4.62) >
prompt_baseline (2.98) >
rrgen_baseline (2.31) > reviewagent_no_spec (2.26)
```

The automatic metrics (BLEU, ROUGE-L, BERTScore) reward responses that closely match the **brief, generic developer replies** in RRGGen’s reference set. Our full-system

condition produces responses $3\times$ longer with $4\times$ the lexical diversity, deliberately introducing IssueSpec-grounded specificity that diverges from the brief reference replies. This divergence is rewarded by human raters (helpful: 92% vs 31%) but penalized by surface-level metrics. **This finding aligns with prior work on the well-documented unreliability of n-gram metrics for response generation [38, 56]**, and provides a concrete empirical instance of where the gap matters in software-engineering applications.

5.3 RAG Without an Issue Specification Is Not Enough

A counterintuitive headline finding from Experiment 2 is that `reviewagent_no_spec` (RAG-augmented, no IssueSpec) scored *lower* than `prompt_baseline` (no RAG, no IssueSpec) on human evaluation: quality 2.26 vs 2.98, helpful% 31 vs 84 (paired Wilcoxon $p < 0.001$, $\Delta = 0.72$). The full system regains those quality points and adds another 1.64 only when the IssueSpec is included (4.62 vs 2.98, $\Delta = +1.64$). The headline reading is **“RAG alone is useless on this task; structure does the load-bearing work.”** Three independent literatures support both halves of this reading.

5.4 Compatibility Class Recovery as a Limitation Becoming a Result

Of the seven categories, `compatibility` initially appeared problematic: only 8 out of 215,583 LLM-labeled reviews were placed in this class, far below the dataset’s true incidence. Rather than suppress this finding, we treat it as a measurable

Table 12: Stage 5 dual-objective design: gap $\rightarrow$ constraint $\rightarrow$ validation. Architecture is complete; empirical validation is at PoC scale and is the highest-priority extension.

	Single-objective RLHF (prior)	Our dual-objective CMDP
Reward	one scalar mixing quality + compliance	quality reward + compliance constraint, separated
Failure mode	quality-vs-compliance trade implicit; safety unreliable at margin	compliance enforced as hard threshold ($C \leq \tau$); quality maximized on remaining surface
Domain prior use	general LM (Safe-RLHF)	safety not applied to app-review response generation
Implementation-		KTO + DPO + Lagrangian Constrained PPO trainers complete (86 unit tests pass); released in <code>src/stage5/</code> (PoC); Llama-3-8B is queued (§7)
Training base	—	distilGPT2 (PoC); head-to-head BLEU/ROUGE/BERTScore (Table ??); active-constraint test <i>not yet possible</i> because distilGPT2 cannot plausibly violate the rubric
Empirical validation	—	

failure mode of LLM annotation. The targeted augmentation — 200 synthetic + 100 keyword-mined samples (95% of which the LLM had originally labeled `bug_report`) — raises V5’s compatibility F1 from **0.00 (V4) to 0.74 (V5 own-test) and 0.83 (vs expert)**. The 100 mined samples themselves provide direct evidence of the LLM’s mislabeling pattern: device-conditioned failures (Samsung-specific crashes, Android version regressions) are systematically forced into the larger `bug_report` bucket.

5.5 Limitations

Single-annotator gold standard. The 490-review expert subset and the 400-row response evaluation were both annotated by the lead author serving as the domain expert. While this is consistent with prior practice in app-review classification [8, 39, 62], it precludes inter-annotator agreement reporting (Krippendorff’s α / Fleiss’ κ). Future iterations of this work should engage 2–3 independent annotators per task to support stronger reliability claims.

Stage 5 RLHF training was implemented but not run. The KTO, DPO, and Constrained PPO trainers (`src/stage5/`) are functionally complete and pass 86 unit tests, but full end-to-end RLHF training was deferred due to compute constraints (RLHF requires multi-GPU infrastructure not available in this study). The architecture-level contribution stands; the empirical comparison of single-objective vs dual-objective RLHF is left to future work.

Conditions 2–4 of Stage 4b use rule-based composers. Only condition 1 (`rrgen_baseline`) was generated via direct LLM-per-response reasoning; conditions 2–4 use deterministic composers parameterized by their condition-specific context. This was a methodological choice to control for LLM stochasticity across the comparison, but it does mean the conditions are not strictly comparable as “same generator, different context.” Reviewers should interpret the conditions as **comparing the value-add of each context source** (system prompt, RAG, IssueSpec) rather than as a head-to-head LLM benchmark. The human evaluation results (which reward the IssueSpec-augmented condition by +2.36 quality points) are robust to this caveat because the human raters score outputs without knowledge of how the outputs were generated.

RRGen as the only application corpus. Our experiments use a single source dataset (RRGen, 58 Android applications). Although RRGen’s review distribution is broad enough to surface the seven Maalej categories, generalization to other app stores, languages, or domains (e.g., desktop software reviews) requires additional study.

The English / lowercase-tokenized review corpus. RRGen reviews come pre-anonymized with placeholders (`<app>`, `<user>`, `<digit>`, `<email>`) and lowercase-tokenized text. This affects readability but, as the human evaluation shows (helpfulness 92% on the full system), is not a barrier to producing useful responses.

Compute constraints throughout. The classifier training (V3, V4, V5) ran on Apple MPS and took 12, 29, and 25 hours respectively per version — far longer than a GPU-equipped workstation would require. This limited iteration speed and informed our choice to use stratified-cap balancing ($\text{cap} = 15,000$ per class) rather than full-corpus training.

Single-LLM dependence — partly mitigated by the §4.2.y cross-LLM run. The headline Stage 3 IssueSpecs and the Stage 4b condition-1 baseline use Anthropic Claude Opus 4.7 [4]. To address the single-LLM-bias concern (), we ran a real cross-LLM replication using the local **Qwen2.5-3B-Instruct** model — the same model already used elsewhere in this paper for aspect extraction (§4.5) and inter-annotator agreement (§4.6) — on a 15-cluster subset. Results in §4.2.y, summarized:

- **Both capable LLMs reach the rubric ceiling on the loose template-fill check** — i.e., the loose-fill score is structurally guaranteed for any reasonably capable instruction-following LLM and is therefore demoted from headline status (§4.2 Table 3 reports only strict numbers).
- **Claude scores 0.971 vs Qwen 0.743 on the strict §3.8.1.x criteria** — a 23-point gap consistent with the expected capability differential between a frontier model and a 3B-parameter local model.
- **Field-level cross-LLM agreement is 71.4%** across 105 strict-fill judgments — moderate, with disagreement concentrated on substantive-content

policy	BLEU-1	BLEU-2	ROUGE-L	BERTScore-F1	mean response length (words)
sft_base (distilGPT2 + 400 SFT samples)	0.0896	0.0142	0.0969	0.8023	53.9
kto_model	0.0682	0.0061	0.0737	0.7923	50.3
dpo_model	0.0839	0.0113	0.0877	0.8000	52.8
constrained_proxy (cross-entropy weighted by quality-safety reward)	0.1369	0.0130	0.1309	0.8064	52.7
lagrangian_ppo (REINFORCE+KL+ λ -dual, 30 steps)	0.0874	0.0149	0.0910	0.7976	55.9

checks (not on whether the LLM follows the template).

- **The qualitative claim** (templated LLM > free-form / human-GitHub on substantive completeness) **holds across both LLMs from different families** (frontier-proprietary Claude vs open-source local Qwen).

The remaining single-LLM concerns — model-specific stylistic biases (Claude’s hedged tone, its safety-training preferences) potentially propagating into Stage 4b’s response generation, and the absence of a third frontier model (GPT-4o, Gemini-Ultra) in the comparison — are not closed by the local-LLM replication. We propose for the camera-ready / journal version (§5.6, §7 item 4): rerun Stage 3 condition (a) and Stage 4b condition (4) on **GPT-4o and Llama-3-70B**, report deltas vs Claude, and verify that the IssueSpec-vs-RAG-only ordering is preserved across all four LLM families. The Qwen-on-15-clusters PoC (§4.2.y) is the **first step** of that program, completed in this paper; the API-based extension is the next.

Three-LLM cross-condition result. §4.2.y now reports a real 3-LLM comparison: Claude Opus 4.7 (frontier proprietary) + Qwen2.5-3B + Qwen2.5-1.5B (within-family scaling, Alibaba). Strict template-fill scales cleanly with capability: $0.97 \rightarrow 0.74 \rightarrow 0.48$. The loose template-fill rate also drops from the rubric ceiling (Claude, Qwen-3B) to 0.93 (Qwen-1.5B), confirming the loose rubric-ceiling result is *not* a universal LLM property — it requires a capable instruction-follower. Two further attempts to add cross-family models (Microsoft Phi-3-mini, HuggingFace SmolLM2-1.7B) hung on this Apple-MPS setup; their scripts are released for GPU re-execution. The completed 3-LLM comparison provides intra-family scaling evidence (3B $\rightarrow$ 1.5B); cross-family frontier replication via GPT-4o + Llama-3-70B + Gemini API is the next step (§7 item 4).

5.6 Validated, Reproducible, Extensible

Three properties of the released pipeline:

Validated. Every numerical claim in §4 is backed by an artifact (script, log, or saved data file) listed in Table D1 below; the third-opinion endorsement of cleanlab corrections by an independently-trained classifier (V5, 88.66%) is itself a validation of the noise-correction methodology. Strict-criteria sensitivity sweeps (§4.2) confirm the qualitative ranking is robust to threshold choice, and cross-LLM replication

(§4.2 + §4.3) confirms the qualitative ranking is robust to LLM choice.

Reproducible. Every result reported in §4 is reproducible from the released artifacts at <https://github.com/Fabiha-9876/ReviewAgent>. Table D1 maps each headline result to its evidence trail; the full pipeline from raw RRGGen to Stage 4b human-rateable output reproduces in ≈ 6 hours wall-clock on a GPU-equipped workstation via a documented sequence of ~ 10 scripts.

Extensible. The pipeline is modular — every stage is a swap-in point. Stage 1 classifier accepts any HuggingFace classifier with the `multi_label_logits` signature. Stage 2 clusterer accepts any `fit_predict`-shaped clusterer (KMeans, BIRCH, OPTICS verified compatible). Stage 3 templates are registered in `src/stage3/taxonomy.py:IssueTaxonomy` and new issue types add by registering a template. Stage 3 LLM is wrapped in `src/common/llm_client.py` (OpenAI / Gemini / vLLM-Llama already stubbed). Stage 4b retriever swaps via `src/stage4b/rag_retriever.py`. Stage 5 RLHF accepts new RL methods by inheriting `feedback_collector.py`. Future researchers can substitute any single component without re-implementing the rest.

Table D1. Claim $\rightarrow$ evidence map.

Replication recipe (single command per stage).

The full pipeline from raw RRGGen to Stage 4b human-rateable output reproduces in ≈ 6 hours wall-clock on a GPU-equipped workstation:

```
scripts/download_datasets.py           # raw RRGGen  $\rightarrow$  Stage 1
scripts/correct_rrgen_v2.py             # V0  $\rightarrow$  Stage 2
scripts/train_classifier_v3.py --v5     #  $\rightarrow$  V5 RoBERTa
scripts/build_frozen_holdout_and_eval.py #  $\rightarrow$  490-review expert
scripts/cluster_phase1b_umap_hdbscan.py #  $\rightarrow$  194 frontier
scripts/run_kg_hierarchical_clustering.py #  $\rightarrow$  605 human
scripts/name_clusters.py                #  $\rightarrow$  TF-IDF
scripts/run_experiments_1_and_2.py      #  $\rightarrow$  Stage 4b
scripts/score_human_work.py             #  $\rightarrow$  400-row
```

Released artifacts inventory: 60+ scripts in `scripts/`, 11 paper-grade figures in `figures/`, 5 trained classifier checkpoints (V1–V5) in `models/`, 5 RLHF policy checkpoints in `data/processed/rlhf/`, the 5,230-review verified anchor, the 490-review expert gold standard, the 400-row blinded human evaluation, 11 evaluation result files. Repository: <https://github.com/Fabiha-9876/ReviewAgent>.

Claim (§)	Headline value	Evidence file / script
LLM mislabels praise (§4.1)	25% on n=5,041	data/processed/expert_evaluation/praise_verification.csv
Cleanlab corrections (§4.1)	44,214 (20.51%)	data/processed/label_issues_v2/corrections.csv
V5 endorses 88.66% of corrections (§4.1)	35,720 / 40,291	scripts/relabel_with_v5.py + log
Cohen κ progression (§4.1)	0.16 $\rightarrow$ 0.33 $\rightarrow$ 0.59	scripts/strict_holdout_kappa.py + 490-review gold
Cluster purity (§4.4)	0.66 / 0.81 (curated)	scripts/score_cluster_validation.py
Hierarchical 605 vs flat 194 (§4.4)	3.1 $\times$ more clusters	data/processed/kg_hierarchical/
Stage 3 strict template-fill 0.959 vs 0.532	substantive content per §3.8.1.x	scripts/recompute_content_validity.py
GitHub (§4.2)		
Faithfulness scores (§4.2)	lexical-overlap proxy	data/processed/issue_specs_5dim/score_specs.py:305
Stage 4b quality 4.62 vs 2.26 (§4.3)	n=400 paired	data/processed/responses/human_ratings.csv
Wilcoxon $\Delta = +2.36$, $p < 0.001$ (§4.3)	paired test	scripts/run_friedman_nemenyi.py
Friedman $\chi^2(3) = 199.3$ (§4.3)	omnibus	scripts/run_friedman_nemenyi.py
Bradley-Terry θ separation > 4.0 (§4.3)	n=498 wins	scripts/run_bradley_tery_mcnemar.py
McNemar 5/6 pairs significant (§4.3)	helpful Y/N	scripts/run_bradley_tery_mcnemar.py
Aspect extraction 84.2% recall (§4.5)	substring micro on n=2,062	scripts/benchmark_aspects_guzman.py
RLHF head-to-head BLEU (§4.7)	100-review test	data/processed/rlhf/head_to_head/metrics.json

6 Conclusion

We have presented **ReviewAgent**, a four-stage pipeline that demonstrates how systematic LLM annotation noise in app-store review datasets can be detected and corrected with a small expert-verified anchor. The contribution is methodological: the verified-anchor + confident-learning correction step turns ≈ 30 person-hours of expert annotation into a leverage point that improves a 215,000-review dataset, with measurable downstream gains across three independent evaluations.

The cleanest empirical signal is the **Cohen κ progression** against expert gold-standard labels: **0.16 $\rightarrow$ 0.33 $\rightarrow$ 0.59** for V2 LLM original $\rightarrow$ cleanlab-corrected $\rightarrow$ V5 trained on corrections. Each pipeline step produces a measurable, externally-validated improvement, and a separately-trained classifier (V5) independently endorses **88.66%** of the corrections — the strongest evidence that the corrections are not artifacts of the procedure.

Two findings have implications beyond app-review classification:

1. **LLM annotation noise is structural, not random.** Class collapse (popular categories absorb minority categories) and boundary confusion (semantically adjacent classes blur together) are predictable failure modes that small expert verification corrects efficiently. The 25% praise mislabeling rate we measure on RRGGen is unlikely to be unique to this dataset.
2. **Retrieval is necessary but not sufficient for paper-grade response generation.** RAG without a structured issue specification underperforms even no-RAG baselines on human evaluation (reviewagent_no_spec quality 2.26 vs prompt_baseline 2.98, $p < 0.001$). Adding the IssueSpec to RAG yields +2.36 quality points ($p < 0.001$) — the structural component is doing the work that RAG alone cannot.

The full-system response generator achieves a **92% helpfulness rate** in a 400-rating blinded human evaluation, against 19% for the original RRGGen-style baseline (a 4.84 $\times$ improvement on identical inputs).

We release all artifacts publicly: 14 scripts implementing the pipeline, 11 paper-grade figures, 5 trained classifier checkpoints (V1–V5), the 5,230-review verified anchor, the 490-review expert gold standard, the 400-row blinded human evaluation, and 11 evaluation result files. The repository is at <https://github.com/Fabiha-9876/ReviewAgent>.

We hope the verified-anchor + confident-learning approach finds use beyond this work — wherever LLM annotation is being used to bootstrap software-engineering datasets at scale.

7 Future Work — Extension Space

The pipeline is structurally open-ended; the limitations identified in §5.5 map directly to a prioritized list of next experiments. We organize them in three tiers (high-priority closes major reviewer concerns; medium-priority closes construct gaps; long-horizon broadens the framework). All items are designed to be backward-compatible with the released code: a future researcher restoring any of these components needs only to swap in the new module at the documented interface (§5.6).

7.1 High-Priority (Closes Major Reviewer Concerns)

1. **Multi-annotator gold standard.** Add 2 independent expert annotators on a 100-review subsample of the 490-review gold standard. Compute Krippendorff’s α and pairwise Cohen’s κ to bound the single-rater bias of all §4 results. This is the highest-leverage near-term item — closes §5.5(A), strengthens the κ progression of §4.1, and enables proper inter-rater reliability in §4.3’s 400-rating evaluation.
2. **Multi-annotator Stage 4b human evaluation.** Recruit 2 additional raters for a 100-row subset of the 400 (review, response) pairs. Re-compute helpfulness, specificity, and quality with proper Krippendorff α and Fleiss κ . This addresses the construct-validity concern of §4.3.3 (rater designed the system).
3. **Full-scale Stage 5 RLHF on a generation-grade base.** Replace the distilGPT2 PoC with

Llama-3-8B-Instruct (or Mistral-7B-Instruct). Use the existing 400-rating preference data for DPO and Constrained-PPO training; collect a held-out 100-row test set; evaluate via Bradley-Terry preference and McNemar safety-violation tests as designed in `src/evaluation/experiment3.py`. This is what would actually test the dual-objective claim of §3.0.

4. **Multi-LLM replication of Stage 3 and Stage 4b.** Rerun condition (a) of Stage 3 and condition (4) of Stage 4b on GPT-4o and Llama-3-70B. Verify the IssueSpec-vs-RAG-only ordering is preserved across LLMs; report any LLM-specific deltas. Closes §5.5 single-LLM dependence.

7.2 Medium-Priority (Closes Construct Gaps)

5. **NLI-based faithfulness.** Replace the lexical-overlap proxy of §3.7.2 with a per-spec-sentence textual-entailment check against the cluster (using DeBERTa-v3-large fine-tuned on MNLI / DocNLI). Compare scores against a 100-spec subset hand-rated for faithfulness on a Likert scale by ≥ 2 raters. This is the construct-correct measurement we currently substitute with a proxy.
6. **Hierarchical purity audit.** Run the same 50-cluster Y/P/N audit on the 605-cluster hierarchical output to confirm H2’s “comparable purity at finer granularity” claim that is currently only provisional.
7. **Anchor-size ablation.** The verified-anchor budget ($5,230 + 5,008 = 10,238$) was chosen pragmatically. Run the cleanlab pipeline at anchor sizes $\in \{1K, 2.5K, 5K, 7.5K, 10K\}$ and report the κ progression as a function of annotation budget. This generalizes the methodology — telling future researchers how much expert annotation actually matters.

7.3 Long-Horizon Extensions

8. **Cross-corpus generalization.** Apply verified-anchor + cleanlab to a second corpus — Apple App Store reviews, Steam game reviews, or a non-English (e.g., Mandarin or Spanish) corpus. Tests whether the noise-correction approach generalizes across review sources, platforms, and languages.
9. **Full agentic resolution (Stage 4a).** Replace the 5-spec stub with a real Planner→Navigator→Editor→Executor loop on open-source Android apps where source repositories are available (e.g., AntennaPod, Signal, Wikipedia). The IssueSpecs from Stage 3 would feed into a SWE-Agent-style loop [1, 44] producing actual patches; success measured via patch-acceptance and test-pass rate.
10. **Closed-loop deployment study.** Deploy the spec-aware response generator to a live app’s developer-response queue. Measure (a) reviewer reply rate, (b) reviewer-rated satisfaction, (c) developer-time-saved

per response. This is the only way to validate the §4.3.3 “helpful” claim against an *outcome*, not a *predicted* outcome.

11. **End-to-end pipeline learning.** Train the classifier, clusterer, issue-specification generator, and response generator jointly with a unified loss that rewards downstream response quality. The current pipeline trains each stage independently; a joint formulation could reveal whether stage-level corrections compound.
12. **Agentic RAG comparison.** Add a fourth Stage 4b condition: an *agentic* RAG loop (multi-turn retrieval, self-refinement, tool use) and benchmark against vanilla RAG and structured RAG. Quantifies whether agentic iteration adds value beyond structured intermediates.
13. **Per-domain template library.** Extend the Stage 3 taxonomy beyond mobile-app issue types — backend incidents (post-mortem template), UX research findings (Hartson template), accessibility issues (WCAG template). Each addition broadens the framework’s domain coverage.

References

- [1] Toufique Ahmed and Premkumar Devanbu. 2024. Few-shot training LLMs for project-specific code-summarization. *arXiv preprint* (2024).
- [2] Eitan Altman. 1999. *Constrained Markov Decision Processes*. CRC Press.
- [3] Enrique Amigó, Julio Gonzalo, Javier Artiles, and Felisa Verdejo. 2009. A comparison of extrinsic clustering evaluation metrics based on formal constraints. *Information Retrieval* 12, 4, 461–486.
- [4] Anthropic. 2025. Claude (Opus 4.7) – Large Language Model. <https://www.anthropic.com/claude>. Used as a writing and formatting assistant during manuscript preparation; see Acknowledgments..
- [5] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [6] Gagan Bansal, Besmira Nushi, Ece Kamar, Walter S. Lasecki, Daniel S. Weld, and Eric Horvitz. 2019. Beyond accuracy: The role of mental models in human-AI team performance. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing (HCOMP)*, Vol. 7. 2–11.
- [7] Tadeusz Caliski and Jerzy Harabasz. 1974. A dendrite method for cluster analysis. *Communications in Statistics* 3, 1 (1974), 1–27.
- [8] Ning Chen, Jialiu Lin, Steven C. H. Hoi, Xiaokui Xiao, and Boshen Zhang. 2014. AR-miner: Mining informative reviews for developers from mobile app marketplace. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. ACM, 767–778.
- [9] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46.
- [10] Mike Cohn. 2004. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional.
- [11] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. 2024. Safe RLHF: Safe Reinforcement Learning from Human Feedback. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [12] David L. Davies and Donald W. Bouldin. 1979. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-1, 2 (1979), 224–227.
- [13] Andrea Di Sorbo, Sebastiano Panichella, Carol V. Alexandru, Corrado A. Visaggio, Gerardo Canfora, and Harald C. Gall. 2016.

- SURF: Summarizer of user reviews feedback. In *Proceedings of the 39th International Conference on Software Engineering (ICSE) Companion*. IEEE.
- [14] Esin Durmus, He He, and Mona Diab. 2020. FEQA: A question answering evaluation framework for faithfulness assessment in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*.
 - [15] Jacek Dbrowski, Emmanuel Letier, Anna Perini, and Angelo Susi. 2022. Analysing app reviews for software engineering: A systematic literature review. *Empirical Software Engineering* 27, 2 (2022), 43.
 - [16] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. 2024. KTO: Model Alignment as Prospect Theoretic Optimization. *arXiv preprint arXiv:2402.01306* (2024).
 - [17] Alexander R. Fabbri, Wojciech Kryciski, Bryan McCann, Caiming Xiong, Richard Socher, and Dragomir Radev. 2021. SumEval: Re-evaluating summarization evaluation. *Transactions of the Association for Computational Linguistics* 9 (2021), 391–409.
 - [18] Joseph L. Fleiss. 1971. Measuring nominal scale agreement among many raters. *Psychological Bulletin* 76, 5 (1971), 378–382.
 - [19] Cuiyun Gao, Jichuan Zeng, Michael R. Lyu, and Irwin King. 2019. Automating App Review Response Generation. (2019), 163–175. doi:10.1109/ASE.2019.00026
 - [20] Cuiyun Gao, Jichuan Zeng, Michael R. Lyu, and Irwin King. 2020. CoRe: Automating review response generation with contextual encoding. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
 - [21] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences (PNAS)* 120, 30 (2023).
 - [22] Max Grusky, Mor Naaman, and Yoav Artzi. 2018. Newsroom: A dataset of 1.3 million summaries with diverse extractive strategies. In *Proceedings of the 2018 Conference of the North American Chapter of the ACL: Human Language Technologies (NAACL-HLT)*.
 - [23] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, and Masashi Sugiyama. 2018. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [24] Rex Hartson and Pardha S. Pyla. 2018. *The UX Book: Agile UX Design for a Quality User Experience* (2 ed.). Morgan Kaufmann.
 - [25] Marti A. Hearst. 1999. Untangling text data mining. In *Proceedings of the 37th Annual Meeting of the Association for Computational Linguistics (ACL)*. 3–10.
 - [26] Or Honovich, Roei Aharoni, Jonathan Herzig, Hagai Taitelbaum, Doron Kukliansky, Vered Cohen, Thomas Scialom, Idan Szpektor, Avinatan Hassidim, and Yossi Matias. 2022. TRUE: Re-evaluating factual consistency evaluation. *Proceedings of NAACL-HLT* (2022).
 - [27] International Organization for Standardization. 2011. ISO/IEC 25010: Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models. Standard.
 - [28] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.
 - [29] Ece Kamar. 2016. Directions in hybrid intelligence: Complementing AI systems with human intelligence. In *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI)*. 4070–4073.
 - [30] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. 2022. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP. *arXiv preprint arXiv:2212.14024*.
 - [31] Klaus Krippendorff. 2004. *Content Analysis: An Introduction to its Methodology* (2 ed.). Sage Publications.
 - [32] Wojciech Krycinski, Bryan McCann, Caiming Xiong, and Richard Socher. 2020. Evaluating the factual consistency of abstractive text summarization. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
 - [33] Philippe Laban, Wojciech Kryciski, Divyansh Agarwal, Alexander R. Fabbri, Caiming Xiong, Shafiq Joty, and Chien-Sheng Wu. 2023. LLMs as factual reasoners: Insights from existing benchmarks and beyond. *arXiv preprint arXiv:2305.14540* (2023).
 - [34] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. 2022. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics* 10 (2022), 163–177.
 - [35] J. Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics* 33, 1 (1977), 159–174.
 - [36] Kuang-Huei Lee, Xiaodong He, Lei Zhang, and Linjun Yang. 2018. CleanNet: Transfer learning for scalable image classifier training with label noise. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 5447–5456.
 - [37] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [38] Chia-Wei Liu, Ryan Lowe, Iulian Serban, Mike Noseworthy, Laurent Charlin, and Joelle Pineau. 2016. How NOT to evaluate your dialogue system: An empirical study of unsupervised evaluation metrics for dialogue response generation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2122–2132.
 - [39] Walid Maalej and Hadeer Nabil. 2016. Bug report, feature request, or simply praise? On automatically classifying app reviews. *Requirements Engineering* 21, 3 (2016), 311–331. doi:10.1007/s00766-016-0251-9
 - [40] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.
 - [41] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 1906–1919.
 - [42] Leland McInnes, John Healy, and Steve Astels. 2017. HDBSCAN: Hierarchical density based clustering. *Journal of Open Source Software* 2, 11 (2017), 205.
 - [43] Leland McInnes, John Healy, and James Melville. 2018. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426* (2018).
 - [44] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. 2450–2462.
 - [45] Jakob Nielsen. 1994. *Usability Engineering*. Morgan Kaufmann.
 - [46] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. 2021. Confident Learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research* 70 (2021), 1373–1411.
 - [47] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank citation ranking: Bringing order to the web*. Technical Report 1999-66. Stanford InfoLab.
 - [48] Nicholas Pangakis, Samuel Wolken, and Nicolaus Fasching. 2023. Automated annotation with generative AI requires validation. *arXiv preprint arXiv:2306.00176* (2023).
 - [49] Giorgio Patrini, Alessandro Rozza, Aditya Krishna Menon, Richard Nock, and Lizhen Qu. 2017. Making deep neural networks robust to label noise: A loss correction approach. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 1944–1952.
 - [50] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Technical Report* (2019).
 - [51] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- [52] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3982–3992.
- [53] Michael V. Reiss. 2023. Testing the reliability of ChatGPT for text annotation and classification: A cautionary remark. *arXiv preprint arXiv:2304.11085* (2023).
- [54] Martin P. Robillard, Andrian Marcus, Christoph Treude, Gabriele Bavota, Oscar Chaparro, Neil Ernst, Marco Aurélio Gerosa, Michael Godfrey, Michele Lanza, Mario Linares-Vásquez, et al. 2017. On-demand developer documentation. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 479–483.
- [55] Peter J. Rousseeuw. 1987. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *J. Comput. Appl. Math.* 20 (1987), 53–65.
- [56] Ananya B. Sai, Akash Kumar Mohankumar, and Mitesh M. Khapra. 2022. A survey of evaluation metrics used for NLG systems. *Comput. Surveys* 55, 2 (2022), 1–39.
- [57] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).
- [58] Sunita Sarawagi. 2008. Information Extraction. *Foundations and Trends in Databases* 1, 3 (2008), 261–377.
- [59] Michael Steinbach, George Karypis, and Vipin Kumar. 2000. A comparison of document clustering techniques. In *KDD Workshop on Text Mining*.
- [60] Alexander Strehl and Joydeep Ghosh. 2002. Cluster ensembles — a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research* 3 (2002), 583–617.
- [61] Tom Tullis and Bill Albert. 2013. *Measuring the User Experience: Collecting, Analyzing, and Presenting Usability Metrics* (2 ed.). Morgan Kaufmann.
- [62] Lorenzo Villarroel, Gabriele Bavota, Barbara Russo, Rocco Oliveto, and Massimiliano Di Penta. 2016. Release planning of mobile apps based on user reviews. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 14–24.
- [63] Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP*. 4195–4205.
- [64] Matt Wynne, Aslak Hellesøy, and Steve Tooke. 2017. *The Cucumber Book: Behaviour-Driven Development for Testers and Developers*. Pragmatic Bookshelf.
- [65] Anonymous Xu. 2025. ReviewGraph: A knowledge graph embedding based framework for review rating prediction with sentiment features. *arXiv preprint arXiv:2508.13953* (2025).
- [66] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [67] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schröter, and Cathrin Weiss. 2010. What makes a good bug report? *IEEE Transactions on Software Engineering* 36, 5 (2010), 618–643.